MINI-COMPILER PROJECT: TECHNICAL REPORT

Phase 1: Lexical Analyzer (Scanner) Architecture & Verification

---

1. Executive Summary

This document provides the formal technical documentation and verification report for Phase 1 of the mini-compiler project. The primary objective of this phase is the implementation of a robust Lexical Analyzer (Scanner) capable of transforming raw source code into a structured sequence of categorized tokens. Built using Python, the scanner incorporates precise line and column coordinate tracking, keyword versus identifier resolution, numeric and string literal classification, comment filtering, and comprehensive error interception.

2. Lexical Architecture & Specifications

The scanner operates on a defined mini-language grammar (`language_spec.txt`). The core processing engine (`lexer.py`) inspects the source input character by character via a lookahead pointer mechanism, guaranteeing linear time complexity *O(n)*.

Token Categories Implemented:

• **Keywords:** Reserved lexemes (`int`, `float`, `bool`, `string`, `if`, `else`, `while`, `return`, `print`).

• **Identifiers:** User-defined names matching pattern [a-zA-Z_][a-zA-Z0-9_]*.

• **Numeric Literals:** Integer numbers and single-precision floating-point numbers containing decimal points.

• **String Literals:** Text enclosed in quotes with validation against unterminated strings.

• **Operators & Punctuation:** Single-character operators (`+`, `-`, `*`, `/`, `=`), multi-character relational operators (`==`, `!=`, `<=`, `>=`), and delimiters (`(`, `)`, `{`, `}`, `;`, `,`).

3. Project File Structure & Component Breakdown

The repository is organized into modular components to support scalability toward Phase 2 (Parser):

File Name	Component Type	Primary Responsibility & Functionality
lexer.py	Core Engine	Implements Lexer and Token classes, scanning logic, state transitions, and error raising.
main.py	Driver Script	Executable entry point supplying sample source code to initialize the lexer and print the token stream.
test_runner.py	Test Suite	Automated harness executing validation checks on valid code and error-handling verification.
bug_report.txt	Documentation	Formal record of test results, error logs, and validation summaries.
language_spec.txt	Specification	Formal definition of mini-language grammar rules, keywords, and token types.

4. Verification & Test Suite Results

A dedicated automated test suite (`test_runner.py`) was executed to validate scanner correctness across normal execution and edge-case errors.

Test 01: Valid Code Tokenization (PASSED)

Input sample testing variable declarations, floating-point parsing, conditional blocks, and print statements.

```
int score = 95; float rate = 3.14; if (score >= 50) { print("Passed successfully!"); }
```

Result: Successfully generated complete token stream with exact line and column indices.

Test 02: Lexical Error Handling (PASSED)

Input sample testing malformed characters and unterminated string literals.

```
int total = 100 @ 50; string msg = "unterminated string;
```

Result: Successfully intercepted illegal character '@' at Line 2, Column 21 and raised precise lexical exception.

5. Conclusion & Next Steps

Phase 1 has been successfully completed, verified, and documented. The scanner provides a clean, stable token stream foundation ready to be consumed by Phase 2 (The Parser) for syntactic grammar validation.